

ComplexGitSync 0002.87– Python API

Nicolas Flipo

September 20, 2026

Contents

1	Direct Python Object API	1
2	Programmatic Project Configuration	2
3	From Empty Registry to READY via .gts	2
4	Object-Level Tag Propagation	3
5	READY Methods in the Client API	3
6	Landing a Branch: <code>merge</code> and <code>refresh_private</code>	5
7	Targeting a Single Path: <code>add</code> and <code>remove</code>	6
8	.gitignore Lifecycle Sync and Commit Identity	6
9	Forcing a Clone Protocol	7
10	Repository Discovery: <code>discover_repos</code>	7
11	Submodule Migration: <code>import_submodules</code>	8
12	Adopting a Submodule Tree: <code>init_from_submodules</code>	8
13	Looking at a memory: <code>memory_status</code>, <code>memory_list</code>, <code>memory_show</code>, <code>memory_explore</code>	9
13.1	Starting a memory’s history over: <code>memory_reboot</code>	10
14	Machine-readable output: <code>status_json</code> and <code>verify_json</code>	11
15	A memory as a repository: <code>memory_init</code>, <code>memory_clone</code>, <code>memory_push</code>	11
16	Register Integrity: <code>verify</code>	12

1 Direct Python Object API

This chapter shows how to manipulate ComplexGitSync objects directly (without the CLI wrapper), including loading a .gts snapshot, reconstructing the runtime registry, mirroring identity into GitTree/GitRepo, and propagating a tag target.

For normal Multi-repo Sync usage, prefer the CLI lifecycle documented in the README and Getting Started guide. The reference test topology is CGSill (https://gitlab.com/CGS_test/CGSill):
initialise -> pull -> checkout/add/commit/push -> freeze -> launch_release.

The lower-level object/API lifecycle remains: `load(.cgs) -> .gts LOADED -> expand(.cgs|.gts) -> .gts PENDING -> validate(.cgs|.gts) -> .gts READY -> pull(.cgs|.gts) -> checkout(.gts) -> add -> git(tree,"commit",msg) -> git(tree,"push") -> git(tree,"tag",name) -> freeze.`

2 Programmatic Project Configuration

Project configuration does not require the CLI or an existing `.cgs` file. The public facade accepts authoring values non-interactively and delegates normalization, static validation, and optional serialization to `cgs_format.py`. It performs no Git or network operations.

```
from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
document = client.configure(
    "GX4G",
    ["codeberg:GX4G/GX4G"],
    output_path="GX4G.cgs", # optional
)
reference_tree = document.to_git_tree()
```

The CLI commands `configure`, `create-cgs`, and `direct initialise -project/-repo` authoring are adapters over this method.

3 From Empty Registry to READY via `.gts`

```
from pathlib import Path

from ComplexGitSync import GitRepo, GitTree, RefKind, TreeLifecycleState
from ComplexGitSync.orchestre import GtsDocument,
    build_registry_from_gts_document
from ComplexGitSync.git_tree import WorkingGitTree

# 1) Empty registry lifecycle
empty_registry = WorkingGitTree()
assert empty_registry.recompute_tree_state() == TreeLifecycleState.
    UNLOADED

# 2) Load the CGSill runtime snapshot (.gts) and rebuild the dependency
    registry
cgshome = Path("../..")
gts_doc = GtsDocument.from_toml(cgshome / ".cgitsync/state/CGSill.gts")
registry = build_registry_from_gts_document(gts_doc)
assert registry.lifecycle_state == TreeLifecycleState.READY

# 3) Rebuild GitTree/GitRepo identity objects from discovered registry
    entries
tree = GitTree()
for entry in registry.values():
    tree.add_repo(
        GitRepo(
            project_owner_name=entry.project_owner_name or "owner",
            project_name=entry.project_name or entry.name,
            gitprovider=entry.gitprovider,
            access_protocol=entry.access_protocol,
            commit_sha=entry.commit_sha,
        )
    )
```

```
)
```

4 Object-Level Tag Propagation

```
# Set the same tag target across all registry entries (parent -> leaves)
tree.propagate_tag(registry, "v1.2.3")
for entry in registry.values():
    assert entry.target_ref_kind == RefKind.TAG
    assert entry.target_ref_name == "v1.2.3"
```

This object-level flow is the same state model used by the CLI and `ComplexGitSyncClient`, but exposed directly for advanced automation and tests.

5 READY Methods in the Client API

The same lifecycle model is exposed by `ComplexGitSyncClient`. In practice, the workflow is:

- `load(...)` is the step-1 name for direct source loading.
- `expand(...)` is the canonical step-2 name; it runs nested `.cgs` discovery from parents to leaves (recursive).
- `validate(...)` is the canonical step-3 name; it accepts both `.cgs` and `.gts` sources.
- `initialise(...)` bootstraps a tree and must finish in `READY`; `clone(...)` is the direct clone entry point for a `.cgs` source; `pull(...)` resynchronises an existing tree.
- `git(tree, command, *args)` is the unified step-5 interface for "commit", "push", and "tag" operations. Each follows leaf-first ordering.
- `freeze(...)` emits the next persisted `.gts` state.

```
from pathlib import Path

from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
cgspath = Path("../..")
cgshome = cgspath / "CGSil1"

# 1. load the CGSil1 test-case .cgs -> .gts LOADED
client.load("../CGSil1.cgs")

# 2. expand(.cgs/.gts) -> .gts PENDING
print(client.expand("../CGSil1.cgs"))

# 3. validate(.cgs/.gts) -> .gts READY
client.validate("../CGSil1.cgs")

# 4. clone(.cgs/.gts)
# Same default as the CLI: output_path defaults to CGSPATH=../..
client.initialise("../CGSil1.cgs")

# Equivalent explicit form:
client.initialise("../CGSil1.cgs", output_path=cgspath)
```

```

# Recovery path for partial/stale clone state:
client.clean_initialise_cgs("../CGSill.cgs", output_path=cgspath)

# Cleanup only:
client.purge_cgs("../CGSill.cgs", output_path=cgspath)

# 5. READY methods - unified git interface
registry = client.get_dependency_registry()
# Pull uses ROOT -> PARENT -> LEAF; every repository in the tree is a
  plain
# independent clone (there are no gitlinks) and receives its own git
  pull.
# A .gts source must be an actual snapshot path (find the current one
  via
# RuntimeStateStore.latest_snapshot_for(), the same lookup the CLI uses
  when
# a command's snapshot argument is omitted) -- unlike a .cgs source,
  there is
# no fixed, predictable filename to hard-code here.
client.pull("../CGSill.cgs")
# pull() fetches every branch of each remote, so checkout() can join a
  branch
# somebody else pushed: it starts the local branch from the remote-
  tracking
# ref when one exists, and only otherwise from the current HEAD.
client.checkout("feature/my-branch")
# Mutations use LEAF -> PARENT -> ROOT.
client.add()
client.git(registry, "commit", "feat: update CGSill CGS#1")
client.git(registry, "push")
client.git(registry, "tag", "v1.2.3")

# 6. freeze -> next .gts id
client.freeze("release-2026.05")

```

Every one of these that reaches a remote folds the memory first. When this tree mounts and has adopted a memory (`.cgitsync/.memory`), `push()`, `tag()`, and `freeze()` each call `memory_push` on it before doing their own work — regardless of `private`, since the memory is this project's own record of itself, not a configuration repository scoped by that flag. `client.last_memory_fold` holds what the most recent one of these folded: the `memory_push` result dict, or `None` when no memory is mounted or the fold could not proceed. A fold that fails — the memory declared but not yet adopted, or unreachable for any other reason — is never raised: it is turned into a `warnings.warn(...)` call instead (catchable with `pytest.warns/warnings.catch_warnings`, the same as `checkout`'s own build-change warning), so a Python caller sees it exactly as a CLI user would, and the write it was folding for still completes. `client.memory_declared()` answers whether a memory is mounted at all, with no side effect, for a caller that wants to know in advance.

`close_branch(branch_name, private=False)`. Renames `branch_name` to `closed/branch_name` across the full tree, leaf-first — locally and on the remote, never deleting anything. Requires `READY`. Raises `GitSyncError` before touching any repository when `branch_name` is the project's own default branch, or when any repository in scope is currently checked out on it — the error names which repository. `client.last_write_outcomes` holds one `RepoOutcome` per repository visited afterward: `acted` is `False` for a repository that had no branch of that name at all, never

an error. `private=True` selects the writable configuration repositories instead of the project's own, the same as `branch()` (create).

```
client.close_branch("memory-dev")
for outcome in client.last_write_outcomes:
    print(outcome.name, outcome.acted, outcome.detail)
```

6 Landing a Branch: `merge` and `refresh_private`

`merge(project_branch, private=False, ff_only=False, no_ff=False)` merges a branch across the tree, leaf-first, and returns one (`repo_name`, `merged_ref`) pair per repository a merge moved.

The argument is always the **project's** branch name. Each repository resolves what that means for itself, so a private/local configuration repository merges `<its default_branch>_<project_branch>` rather than a branch of the project's name, which it does not have. `merge_plan(project_branch, private=False)` returns one (`repo_name`, `source_ref`, `status`, `conflicting_paths`) row per repository without merging anything — what the CLI's `-dry-run` prints. `status` is "merge", "already-on-it", "no-branch" or "conflicts", and the last one carries the files that block it.

Every repository in scope is checked before any is merged, so a conflict anywhere leaves the whole tree untouched. The check names the conflicting files, and counts binary files: a tracked PDF changed on both sides conflicts just as a source file does.

`merge_resolve(project_branch, private=False, ff_only=False, no_ff=False)` is the opt-in alternative. It merges one repository at a time, stops at the first that conflicts, and returns a `ResolveOutcome` naming what it merged, where it stopped, the files that stopped it, and the repositories it never reached. It gives up the all-or-nothing guarantee on purpose: that is the only way to reach the conflicted worktree a merge tool needs.

`open_merge_tool(repo_name)` then opens one repository's conflicts in a merge tool. It returns `None` once the tool has run, or the command to run by hand when no tool is available — a missing editor is a normal outcome, not an error. A `merge.tool` you configured always wins, and your Git configuration is never written.

`refresh_private()` is the other direction, and what `pull -private` runs: it fetches and merges each private/local repository's base branch into the derived branch it is on, so a settings branch does not drift behind the project's while a feature branch is open.

```
client.merge("multi-branch")           # the project's own repos
client.merge("multi-branch", private=True) # ComplexGitSync_multi-
    branch                               # -> ComplexGitSync
client.merge("multi-branch", all_writable=True) # both halves, one
    pass

# Merging into a branch that is not checked out, in one call.
client.merge_into_plan("multi-branch", "main") # what it would do, per
    repo
client.merge_into("multi-branch", "main")      # checkout and merge,
    together
client.build_installed_from("main")            # the version that
    branch carries
client.refresh_private()                       # the other way round
```

all_writable: the API half of `-all`. `add`, `commit`, `push`, `merge`, `merge_resolve` and `merge_plan` each take `all_writable=False` alongside `private=False`. Setting it runs the call

across this project's own repositories *and* its writable configuration ones in a single pass. The two are mutually exclusive and passing both raises; leaving both unset keeps today's behaviour exactly. Read-only configuration repositories are never written to under any combination — the parameter maps to `RepoScope.WRITABLE`, which is narrower than `RepoScope.ALL`.

7 Targeting a Single Path: add and remove

`add(paths=None)` and `remove(paths)` accept one or more filesystem paths (each relative to `CGSHOME`, or absolute) instead of operating on the whole tree:

```
# Stage only these two files, each in the repo that owns it -- every
# other repo in the tree is untouched.
client.add(paths=["notes.md", "deps/leaf/src/main.py"])

# Remove a single tracked file: deleted from disk, removal staged.
client.remove(["deps/leaf/obsolete.txt"])
```

Both resolve each path to its owning repo through the same helper, `resolve_repo_for_path(tree, path)` (`git_tree.py`) — the deepest (most specific) repo whose `absolute_path` contains the resolved path wins, so a nested child is correctly preferred over its parent. A path outside every repo in the tree raises `GitSyncError` immediately, before anything is staged or removed; for `remove`, a path that resolves to a directory or does not exist raises the same way rather than partially applying. `add()` with no `paths` keeps today's exact behaviour: every repo staged in full, leaf-first.

`remove` is a plain `git rm` (deletes from disk, stages the removal) — unrelated to `GitRunner.rm_cached` (index-only, used internally by `import_submodules` to drop a gitlink while keeping the child's working tree); the two are not interchangeable.

8 .gitignore Lifecycle Sync and Commit Identity

`initialise_cgs`, `initialise_cgs_document`, `clean_initialise_cgs`, `clean_init`, `restart`, and `pull` (for `.cgs` sources) all accept the same four optional keyword arguments, mirroring the CLI's `-commit-gitignore`, `-force-gitignore-sync`, `-git-user-name`, and `-git-user-email` flags:

```
from ComplexGitSync import ComplexGitSyncClient, MasterConfig

client = ComplexGitSyncClient()
client.initialise_cgs(
    "../CGSill1.cgs",
    output_path=cgspath,
    commit_gitignore=True,          # stage/commit/push each changed .
    gitignore                      # gitignore
    force_gitignore_sync=False,    # pull-force fallback for a blocked
    repo
    git_user_name="cgitsync-bot",
    git_user_email="bot@example.com",
)

# The identity above is persisted to CGSHOME/.cgitsync/master.toml, so a
# later call on the same workspace picks it up without repeating it:
client.initialise_cgs("../CGSill1.cgs", output_path=cgspath,
    commit_gitignore=True)
assert MasterConfig.resolve_identity(cgshome, client.git_runner) == (
    "cgitsync-bot",
```

```

        "bot@example.com",
    )

```

All four keyword arguments default to `False/None`: by default, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled and has its `.gitignore` updated, but nothing is staged, committed, or pushed, and the commit identity — were a commit ever made — would be whatever `git config user.name/user.email` already resolves to locally. `MasterConfig` (`master.py`) is the class behind the identity override: `configure()` sets it in memory for the current process, `persist(cgshome, ...)` writes it to `CGSHOME/.cgitsync/master.toml` (and also updates the in-memory value), `load(cgshome)` reads a previously persisted override back in, and `resolve_identity(repo_path, git_runner)` returns the `(user_name, user_email)` pair — or `(None, None)` to defer entirely to local git config. This file is workspace-local configuration, not part of the `.cgs/.gts` project spec, and is preserved (not deleted) by `purge/clean-init`.

9 Forcing a Clone Protocol

`initialise_cgs`, `initialise_cgs_document`, `clean_initialise_cgs`, `clean_init`, `bootstrap`, and `clone_cgs` additionally accept `force_access_protocol`, mirroring the CLI's `-force-protocol` flag on `initialise`, `bootstrap`, and `clean-init` (`restart/pull` do not accept it):

```

# Every repo this run clones -- root siblings and any child discovered
# later from a nested .cgs in a different repo -- is cloned over https,
# regardless of what its own .cgs entry says. No .cgs file is read
# differently or written.
client.initialise_cgs(
    "../CGSill.cgs",
    output_path=cgspath,
    force_access_protocol="https",
)

```

"ssh" or "https"; `None` (the default) leaves every entry's own `.cgs`-declared `access_protocol` untouched. The override lives only in memory for the duration of the call — it is applied at the single point every clone's remote URL is built (`ComplexGitSyncClient._build_remote_url`), so it reaches root siblings and nested-discovered children alike without `cgs_format.py` parsing, or any `.cgs` file on disk, ever being touched differently.

10 Repository Discovery: `discover_repos`

`discover_repos(root_dir=None, *, max_depth=5, output=None)` scans a directory for git repositories already checked out on disk and drafts a `.cgs` from what it finds — the entry point for adopting a project that exists but has no specification yet:

```

from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Read-only scan - nothing is cloned, fetched, or modified
report = client.discover_repos("/path/to/checkout")
for repo in report.repos:
    print(repo.relative_path, repo.identifier, repo.branch)

for warning in report.warnings:
    print("unresolved:", warning)

```

```
# Write the draft out (same call, with an output path)
report = client.discover_repos("/path/to/checkout", output="draft.cgs")
```

The method returns a `DiscoverReport` dataclass with fields `root` (the scanned directory), `repos` (a tuple of `DiscoveredRepo`), `cgs_entries` (authoring-form repo tables ready for `configure()`), `warnings`, and `project_name`. Each `DiscoveredRepo` carries `relative_path`, `absolute_path`, `remote_url`, `identifier`, `branch`, and `has_cgs`.

Repository addresses are resolved through the existing `parse_repo_id()` grammar; a repository with no `origin` or an unrecognised provider is reported in `warnings` and omitted from `cgs_entries` rather than guessed at. Passing `output` raises `GitSyncError` when nothing resolvable was found, so an empty draft is never written.

11 Submodule Migration: `import_submodules`

`import_submodules(repo_root, *, apply=False, recursive=False)` converts git submodules in a local repository to plain `ComplexGitSync` nested clones:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Dry run - inspect without changing anything
report = client.import_submodules("/path/to/repo")
for sub in report.submodules:
    print(sub.name, sub.path, sub.url)

# Apply - convert every level, root first
report = client.import_submodules(
    "/path/to/repo",
    apply=True,
    recursive=True,
)
print(report.converted)    # tuple of converted submodule names
```

The method returns an `ImportSubmodulesReport` dataclass with fields `submodules` (all entries found), `applied` (whether the conversion ran), `converted` (names of converted submodules), and `scan_root` (the repository the call was pointed at, which every reported path can be expressed from). When `apply=False` (the default), no files are modified. When `apply=True`, each submodule is removed from the index (`git rm -cached`), its stanza is removed from `.gitmodules`, and the parent's `.gitignore` is updated to ignore the child path. The staged changes should then be reviewed and committed manually.

`recursive=True` also converts any submodule that has its own checked-out `.gitmodules`, at any depth. That walk follows the submodule graph declared by `repo_root`'s own `.gitmodules`, so a repository whose holder has already been converted is no longer reachable — which is why the conversion of a whole tree happens once, and last (see `init_from_submodules` below).

12 Adopting a Submodule Tree: `init_from_submodules`

`init_from_submodules(repo_root, *, cgs_path=None, max_depth=5, dry_run=False, force=False, force_access_protocol=None)` performs the whole adoption of a submodule-based checkout, in the one order that works:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
```

```

# Report the plan without writing, cloning, or converting anything
plan = client.init_from_submodules("/work/project", dry_run=True)

# Adopt: discover -> write .cgs -> initialise -> convert every submodule
report = client.init_from_submodules("/work/project")
print(report.cgs_path)                # the .cgs used
print(report.tree.is_ready())         # True
print(report.import_report.converted) # converted submodule names

```

Each step delegates to the method that already owns it (`discover_repos`, `initialise_cgs`, `import_submodules`); the value this method adds is the ordering and the preconditions. `initialise_cgs` adopts the root in place but deletes and re-clones every other repository from its remote, where the submodules are still declared — so converting first is undone for every non-root repository, and a repair pass cannot reach any deeper level once the root's `.gitmodules` is gone.

The method returns an `InitFromSubmodulesReport` with fields `root` (which is also `CGSHOME`), `discover` (the `DiscoverReport` from step 1), `cgs_path`, `cgs_written`, `import_report`, `tree`, and `dry_run`. Nothing is committed: the conversion touches every repository that held a submodule, so `branch`, `checkout`, `add`, and `commit` stay explicit, separate calls. A `GitSyncError` is raised before anything is written when the discovery resolved nothing, when the project name does not match the directory name (`CGSHOME` is `<parent>/<project-name>`), or when `repo_root` has no `.gitmodules` to convert (`force=True` overrides the last one).

13 Looking at a memory: `memory_status`, `memory_list`, `memory_show`, `memory_explore`

The four read-only questions the CLI's memory group asks, each returning plain data rather than text:

```

from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

status = client.memory_status("/path/to/CGSHOME")
status["states"], status["entries"]          # how much is remembered
status["verification"] # verified / no-history / legacy / corrupt /
    time-inconsistent
status["latest_toolchain"]["git"]            # what produced the newest
    record

for row in client.memory_list("/path/to/CGSHOME"):
    row["state"], row["recorded_at"], row["commands"]

state = client.memory_show("/path/to/CGSHOME", "2acdc98b") # hash or
    prefix
state["repos"], state["entries"]
for entry in state["entries"]:
    for commit in entry["commits"]:
        commit["message"], commit["repository"], commit["published"]
state["published"] # remote and ref each commit
    went to

answer = client.memory_explore("/path/to/CGSHOME")
answer["branch"] # the memory branch checked out
    here, or None

```

```

for commit in answer["commits"]:          # published commits, newest
    push first
    commit["repository"], commit["branch"], commit["sha"], commit["
        message"]

timeline = client.memory_explore("/path/to/CGSHOME", timeline=True)
for entry in timeline["entries"]:        # every ledger entry, in order
    entry["seq"], entry["command"], entry["commits"], entry["published"]

```

Commit messages come back whole. Shortening a long one to a single line is the caller's decision, which is why the CLI has `-full` and this method does not: a caller reading the memory from Python wants the message that was written, not the one that fitted.

`memory_show` raises `GitSyncError` when the prefix matches no State, or more than one.

`memory_explore`'s default view answers what a colleague pulling this branch would see — commits with no publication row are left out, because that is `memory_show`'s question, not this one. `branch=` asks for a memory branch other than the one checked out on this disk; today only the checked-out one can be read this way, and a mismatch raises `GitSyncError` naming `memory_clone(branch=...)` rather than answering for the wrong branch.

The commands that give a memory a repository of its own have the same mirror. `add_memory_repo_cgs` is the one the owner named; the CLI spells it `memory mount`.

```

client.repo_create("github:you/.memory", private=True)
# {"created": "created" / "exists" / "unavailable", "command": "gh
  repo create ..."}

client.add_memory_repo_cgs("project.cgs")          # one entry, comments
    kept
client.memory_adopt("/path/to/CGSHOME")            # .cgitsync/.memory,
    fresh and empty
client.memory_push("/path/to/CGSHOME")             # folds .cgitsync's
    pending content in first
client.memory_branch("/path/to/CGSHOME", "main")   # the branch a merge
    will need

# Only for a memory mounted before this command existed, directly at
# .cgitsync instead of .cgitsync/.memory:
client.memory_migrate("/path/to/CGSHOME", "project.cgs")

```

`repo_create` never raises for a repository that already exists — that is "exists", an ordinary answer — and never for a provider tool that is absent or signed out, which is "unavailable" carrying the command to run. It raises only for a provider no tool is known for, and for a tool that failed for some other reason. `memory_list` includes a State that entries name but that is not on disk, with `path` set to `None` — `verify` reports it as `MISSING_STATE`, and this is how a caller finds out which one.

13.1 Starting a memory's history over: `memory_reboot`

```

answer = client.memory_reboot("/path/to/CGSHOME")
answer["folded"]          # pending records folded in before archiving
answer["archived_from"]   # the branch's name before the reboot
answer["archived_to"]     # "<branch>.archived-<YYYYMMDD>", real, on
    origin
answer["exported"]        # .cgitsync/.memory/.cgs/<project>-v<N>.cgs
answer["branch"]          # the original name, fresh, empty, checked out

# A fresh start from the very first adopt, instead of inheriting
    whatever

```

```
# the fallback branch already holds:
client.memory_adopt("/path/to/CGSHOME", remote=remote, branch=branch,
    reboot=True)
```

Nothing is ever force-pushed or deleted: the old branch is pushed to origin under its archived name *before* the old name is removed from origin, so the commits it holds are reachable under some name on the remote for the whole operation, never only locally. `memory_reboot` raises `GitSyncError` when the mount is not a repository yet, and when today's archived name already exists — a second reboot the same day needs a name of its own rather than colliding with the first.

14 Machine-readable output: `status_json` and `verify_json`

Two methods return the same answers as `status()` and `verify()`, serialised as one JSON object each. They are what the CLI's `-json` flag prints, so a Python caller and a shell pipeline read exactly the same fields.

```
import json
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/snapshot.gts")

payload = json.loads(client.status_json())
payload["cgitsync_branch"]           # the branch the tree is on
payload["summary"]["repos"]          # how many repositories it holds
payload["repositories"][0]["sync"]   # per repository, as the table shows
it

chain = json.loads(client.verify_json("/path/to/CGSHOME"))
chain["status"]                      # "clean" or "findings"
```

`verify_json` keeps the report it built on `client.last_verify_report`, so a caller needing both the object and the verdict does not verify the chain twice — which under `repair=True` would mean repairing twice.

The shape of both objects lives in `json_render.py`, defined once for every command rather than per command, and carries a `schema_version`. The promise attached to it is **additive only**: new fields may appear; existing fields do not change meaning and do not vanish.

15 A memory as a repository: `memory_init`, `memory_clone`, `memory_push`

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/CGSHOME/.cgitsync/state/<hash>.gts")

proposal = client.memory_init("/path/to/CGSHOME")
proposal["entry"]           # the .cgs entry that mounts the memory
proposal["branch"]          # this project's branch of the memory
    repository
proposal["create_with"]     # the command that creates it -- never run for
    you

client.memory_push("/path/to/CGSHOME")           # commit +
    push
```

```
client.memory_clone("/other/workspace", owner="flipoyo")    # onto a new
machine
```

`memory_clone` needs no loaded project — that is the case it exists for — so give it an **owner** and a **branch** when nothing is loaded, or a **remote** address directly for a memory that does not live where the convention says. It raises `GitSyncError` rather than writing over a memory already present, or when the branch it wants has never been pushed.

Calling `memory_push` by hand, as above, is for settling the memory on its own. *READY Methods in the Client API*, earlier in this chapter, covers the more common case: `push`, `tag`, and `freeze` each call it automatically, first, once a memory is mounted and adopted.

16 Register Integrity: `verify`

`verify(cgshome, *, repair=False)` checks the hash-chained `.cgitsync/lgr` register under `cgshome` for tamper-evidence:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
report = client.verify("/path/to/CGSHOME")

if report.is_clean:
    print("register is clean")
else:
    for seq, finding, detail in report.findings:
        print(seq, finding.name, detail)

# Repair a stale HEAD cache (never rewrites or deletes an entry)
client.verify("/path/to/CGSHOME", repair=True)
```

Returns an `integrity.VerificationReport`. Read `report.state` first: it is one of four `HistoryState` members — `VERIFIED`, `NO_HISTORY`, `LEGACY`, `CORRUPT` — and `report.is_verified` is `True` only for the first.

`report.is_clean` is a narrower question: it means “no findings were recorded”, which an empty register also satisfies. Use it to ask whether anything is *wrong*; use `state` to ask what was actually established. `findings` holds `(seq, Finding, detail)` tuples.

Every command that writes a `State` now appends an entry to `.cgitsync/lgr/`, so a workspace used since that landed answers `VERIFIED`. A workspace whose only history predates it answers `LEGACY`: the single-file register is still read, and nothing writes it any more. `Finding` enumerates `BROKEN_LINK`, `BAD_ENTRY_HASH`, `SEQ_GAP`, `SEQ_DUPLICATE`, and `HEAD_STALE` for what this method actually checks (chain linkage and the `HEAD` cache); its remaining three members (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`) are reserved for a future store-level verification pass that cross-references entries against the actual `state/<hash>.gts` files on disk.

With `repair=True`, a stale `HEAD` cache is corrected in place. Register entries are never rewritten or deleted by this method, repair or not — a broken chain is reported, not silently healed.